Toxic Comment Detection using LSTM

Krishna Dubey
Department of Information Technology
Thadomal Shahani Engineering College
Mumbai, India
krishnadubey1997@gmail.com

Rahul Nair
Department of Information Technology
Thadomal Shahani Engineering College
Mumbai, India
nair.rahul207@gmail.com

Mohd. Usman Khan
Department of Information Technology
Thadomal Shahani Engineering College
Mumbai, India
uk80718@gmail.com

Prof. Sanober Shaikh
Department of Information Technology
Thadomal Shahani Engineering College
Mumbai, India
s.sanober1@gmail.com

2020 Third International Conference on Advances in Electronics, Computers and Communications (ICAECC) | 978-1-7281-9183-6/20/$31.00 ©2020 IEEE | DOI: 10.1109/ICAECC50550.2020.9339521

***Abstract*—While online communication media acts as a platform for people to connect, collaborate and discuss , overcoming the barriers for communication, some take it as a medium to direct hateful and abusive comments that may prejudice an individual's emotional and mental well being. Explosion of online communication makes it virtually impossible for filtering out the hateful tweets manually, and hence there is a need for a method to filter out the hate-speech and make social media cleaner and safer to use. The paper aims to achieve the same by text mining and making use of deep learning models constructed using LSTM neural networks that can near accurately identify and classify hate-speech and filter it out for us. The model that we have developed is able to classify given comments as toxic or non-toxic with 94.49% precision, 92.79% recall and 94.94% Accuracy score.**

***Index Terms*—Hate Speech, Word Embedding, Artificial Neural Networks, LSTM, NLP**

I. Introduction

Diverse communities such as education, social, business, political, etc. show a great interest in internet based collaborative environments with a desire to share valuable contents with others, to grow and nourish relationships, to get the word out about brands and causes they like or support, and many more. Online communication media is being used in ways that shape politics, business, world culture, education, careers and innovation. Flood of information is produced everyday through the global internet usage arising from the widespread use of online interactive communication media. While this situation contributes significantly to the quality of human life, unfortunately it involves dangers like texts with high toxicity that can cause personal attacks, online harassment and bullying behaviors that may prejudice an individual's emotional and mental well being. The threat of abuse and harassment online force many people to stop expressing themselves and give up on expressing different opinions.

Strong words are used sometimes in different situations not necessarily as a verbal abuse but rather as an exclamation and such has always been a part of natural language. Such forms of expression if written in the text cannot be classified necessarily as a toxic comment but if toxic words are used to insult someone, shame them, or make them feel bad then that piece of text can be classified surely as toxic. To address concerns about offensive and hate content over the internet, social media administrators often manually review online contents to detect and delete offensive materials. Manual reviewing of offensive contents are however labour intensive, time consuming, and thus not sustainable and scalable in reality. Some content filtering softwares such as Appen and Internet Security Suite, have been developed to detect and filter online offensive contents. Most of them simply block web pages and paragraphs that contain dirty and offensive words. Online communication platforms struggle to effectively facilitate healthy conversations, leading many communities to limit or shut down user comments.

In this project we are trying to explore how sentiment analysis using deep neural networks can help us identify and classify toxic texts in online communication systems and try and filter these out in the best possible way so that we can make internet communication media cleaner and safer to use.

II. Domain Overview

A. Hate Speech

Hate speech is defined by Cambridge Dictionary as "public speech that expresses hate or encourages violence towards a person or group based on race, religion, sex or sexual orientation, country, caste and creed". Hate speech is any form of communication may it be verbal, written or gesture based on various aspects such as race, color, gender, disability, citizenship and religion which is used to provoke unpleasant feelings in an individual or group of individuals that may incite violence, self-harm or any harm to the public in general. Verbal and textual speech is a powerful medium of communication that has a massive impact on almost every aspect of life and is the widely used mode of communication. Words and sentences usually carry a lot of contextual and emotional value. Hence a word accompanied with negative and vulgar emotions, such as abusive words if used in a conversation is usually accompanied with emotion of hate and insult in order to demean the entity to

978-1-7281-9183-6/20/$31.00 ©2020 IEEE

Authorized licensed use limited to: IEEE Xplore. Downloaded on May 15,2021 at 08:15:50 UTC from IEEE Xplore. Restrictions apply.

whom it is directed towards, can be identified as toxic speech. Although criticism can be a part of conversation, constructive criticism is always encouraged and is a better way to express an unpleasant feeling, rather than insulting the other person.

B. Natural Language Processing (NLP)

Natural Language Processing, abbreviated as NLP is a branch of Artificial Intelligence that deals with processing human understandable language and making machines understand it. The main goal of NLP is to extract, process and understand the human languages to make sense out of it. Most of the NLP techniques rely on machine learning algorithms to process data and get valuable insights out of it. Some applications of NLP are Sentiment analysis, personal assistant applications like Siri, Alexa, Google, etc. Language translator, etc. The job of NLP is to extract natural language rules from the given piece of text and convert it to computer understandable languages. This is achieved by applying various algorithms to extract the meaning of every sentence in text and collecting essential data from it. Human languages contain words which have emotions attached to them. The same sentence can have different meanings when used in different contexts. Therefore, it is necessary for computers to understand and interpret the meaning of a given sentence in accordance to its context. There are various steps of NLP in which the text processing is done. Following are some general steps of NLP known:

- Lexical Analysis:
  Tokenizing text up to word level for identifying and analyzing the structure of those words.
- Syntactic Analysis:
  Analyzing the words in a sentence with respect to grammar and arranging them in a manner that shows relationship among the words. For example "A home goes person to" is rejected because the given phrase is grammatically incorrect. Although the phrase, "Sita has a moustache" cannot be meaningful, it will not be rejected because the arrangement of words grammatically is correct.
- Semantic Analysis:
  Here analysis of exact meaning or dictionary meaning of text is done. For example, the phrase, "Sita has a moustache" will be rejected because as per dictionary and human language considered "Sita" is a girl and a girl having a moustache is meaningless.
- Discourse Integration:
  As mentioned above, the same sentence can have different meaning when written in different contexts. Therefore, it is necessary to analyze a sentence before the given sentence to get the context. It also helps in deciding the context of proceeding sentence
- Pragmatic Analysis:
  Here a sentence is analyzed based on real world knowledge. For example in a phrase, "Sita has a moustache" it is assumed that "Sita" is female. But given the conditions wherein "Sita" is the name of a male person.

C. Artificial Neural Network

An artificial neural network consists of numbers of artificial neurons arranged in a particular fashion to get some work done. A simple artificial neuron mimics the structure of biological neurons. A biological neuron is as shown in following figure:

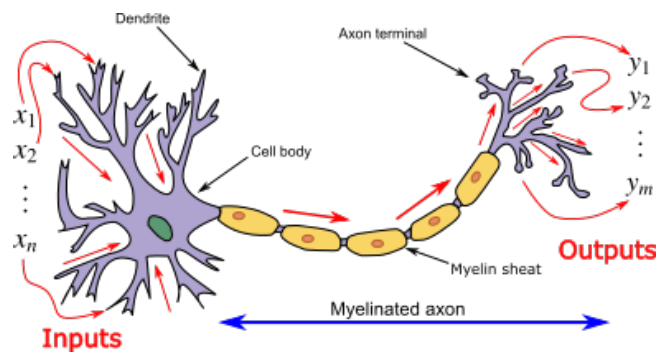

Fig. 1. Structure of Brain Neuron

- Dendrites: Acts as input to the neuron, responsible for receiving the information from other neurons it is connected to.
- Soma: The body of biological neurons which is responsible for processing of information received from dendrites.
- Axon: A path through which neurons transmits the information.
- Synapses: A very small (20nm) gap between the axon and other neuron dendrites.

An artificial neuron is made on similar lines which is as shown in following figure:
Here x1, x2, ..., xm are the inputs to the artificial neuron

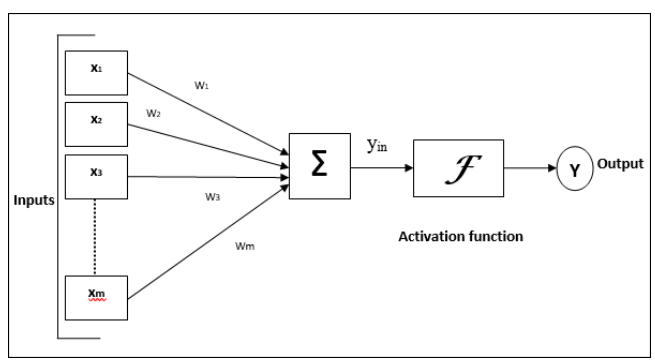

Fig. 2. Structure of Artificial Neuron

similar to the dendrite in biological neurons and w1, w2, ..., wm are the weights assigned to the respective inputs of an artificial neuron which does the work of synapses in biological

Authorized licensed use limited to: IEEE Xplore. Downloaded on May 15,2021 at 08:15:50 UTC from IEEE Xplore. Restrictions apply.

neurons. Sigma takes all the inputs and weights and calculates the sum of products as x1w1+ x2w2+ .... + xmwm. The calculated sum of product is then passed to an Activation Function which can be any mathematical function (eg. sigmoid function) which produces output to be passed to the proceeding neuron or can be the final output.

Such artificial neurons are arranged in a layer wise manner to form an artificial neural network. Neural networks work on a trial-and-error basis, therefore they need massive amounts of data to train on. Ater feeding the data the training process starts with forward pass. In forward pass, information is fed into the input layer which propagates forward through the network and produces output values. We compare the output values with our expected results. Next, we calculate the errors and propagate the information backward to train the network and update the weights. This process is called Backpropagation. When we have adjusted the weights to the optimal level (such that we get the expected output), we are ready to proceed to the testing phase. Depending on the type of arrangement of neurons and the activation function used we have several types of artificial neural networks to be used in several different applications. All these neural networks function in somewhat similar ways, by feeding data in and letting the model figure out for itself whether it has made the right interpretation or decision about a given data element.

For this project we used the Long-Short Term Memory(LSTM) neural network which is a type of Recurrent Neural Network(RNN). We don't choose to use simple RNN for this project because it suffers through a problem of vanishing gradient and exploding gradient and for better accuracy, our project requires a neural network which should not suffer from vanishing gradient problem. So what is the vanishing gradient problem? When we do Back-propagation and calculate change

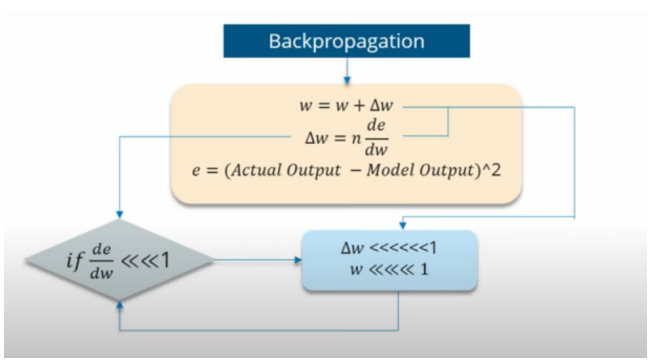

Fig. 3. Vanishing Gradient problem in RNN

in error with respect to the weights (i.e. de/dw), the value turns out to be very very small and the learning rate i.e. "n" here is already less than 1 therefore change in weight (i.e. w) tends to get smaller and smaller. Therefore the new weight obtained is almost the same as the previous one because overall change in weight is found to be very very small. Therefore the neurons in the initial layers learn very slowly as compared to the neurons in the later layers in the sequence. Therefore the initial layers in the network are slowest to train. Hence the training process takes too long and the Prediction Accuracy of the Model will decrease.

D. Long-Short Term Memory(LSTM)

It is a type of Recurrent neural network with extended features (mathematical functions and cell state in particular fashion) to overcome the problem of vanishing and exploding gradients. Due to the additional feature the LSTM network gets advantage of memory. In RNN only the previous prediction was given as input with the new input therefore it was only able to remember short sequences. But due to the availability of memory in the LSTM network it stores and remembers the long sequences also.

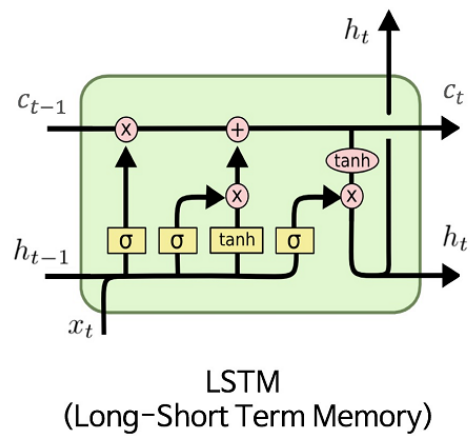

Fig. 4. Long Short Term Memory

The above diagram shows the repeating module of LSTM. Here xt is the actual input at time t, ht-1 is previous prediction (i.e. prediction at time t-1) and ct-1 is the cell state(the key idea behind LSTM) output of the previous repeating module. ht-1 and xtare concatenated and provided as input to the activation functions in the diagram. is the sigmoid function which gives output between 0 and 1, tanh is hyperbolic tan function which gives output between -1 and 1. Pointwise operation like vector addition takes place in the pink coloured circles. All the operations are performed and again the prediction and cell state output is produced at time t.

III. System Design and Implementation

A. Design Overview

The design of the toxic text classifier model proceeds as depicted in the below flow diagram. Massive dataset consisting of 56745 tweets and comments has been considered for training the model, so that the model can train on the real world communication data. The dataset has been labelled appropriately as toxic and non toxic based on the language used and the nuances considered. The tweets have been thoroughly preprocessed by making use of NLTK library in Python. Stopwords are removed to retain the most meaningful words, Lemmatization has been performed for extracting the

Authorized licensed use limited to: IEEE Xplore. Downloaded on May 15,2021 at 08:15:50 UTC from IEEE Xplore. Restrictions apply.

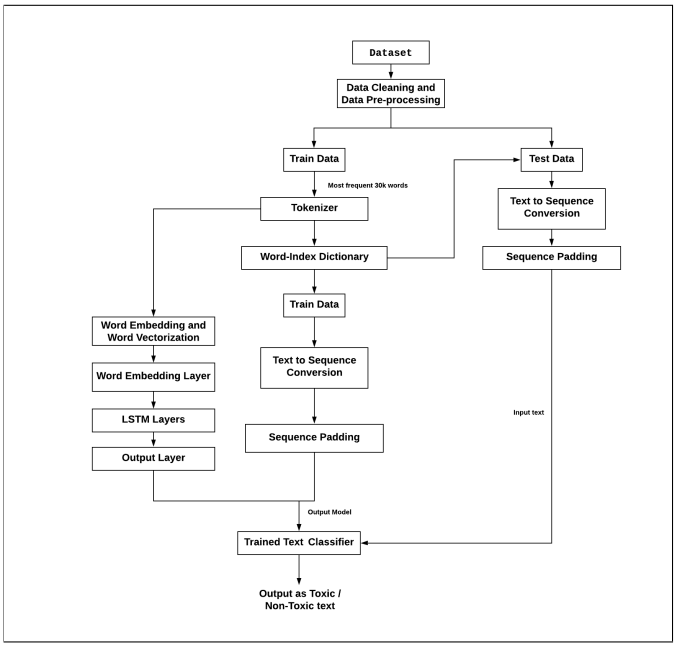

Fig. 5. Project Flow Diagram

essence word from its various forms, POS tagging has been used for disambiguation of words and regular expressions are used to remove punctuations, hyperlinks, hashtags and twitter mentions, convert words from upper case to lower case to avoid difference in consideration of same words due to case sensitiveness. Also some commonly used short forms of words are converted back to their correct english forms, thus giving a much cleaner dataset for the model to train on. After the data cleaning dataset is first split into train and test data, tokenization is done on train data to split sentences into an array of words. This is required for word vectorization as neural networks only train on numeric data. SpaCy word embeddings are used to generate the word embeddings for the most frequent 30000 words in the text. LSTM is used to train the model with appropriate dropouts considered to prevent overfitting and improve accuracy.

B. Implementation

1) Analysing Dataset: Two separate datasets were downloaded from Github. The first file had a tweet column and three classification categories viz. Hate-speech, toxic-speech, neither. The second file had a tweet column and a single classification category called "class" with two values "0" which represents non-toxic text and "1" which represents toxic text. The second csv file was a collection of tweets and contained more non-toxic tweets than toxic tweets whereas the first file was a collection of texts from various sources that had more toxic texts and lesser non-toxic texts. Now the categories in 1st file was narrowed down into a single category called "class" wherein if the tweet belonged to hate-speech or toxic speech then the class value would be "1" else "0". And then the datasets were merged into a single dataset and stored.

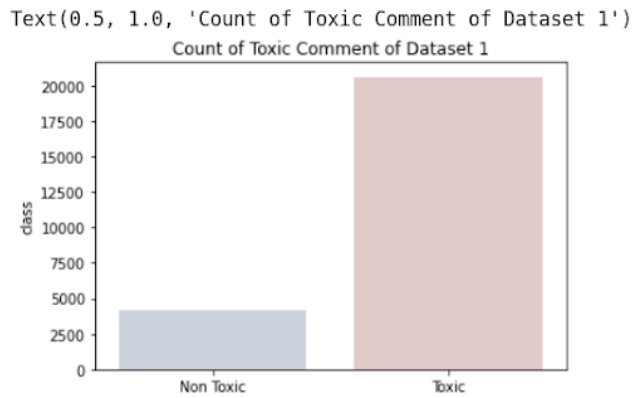

Fig. 6. Dataset 1

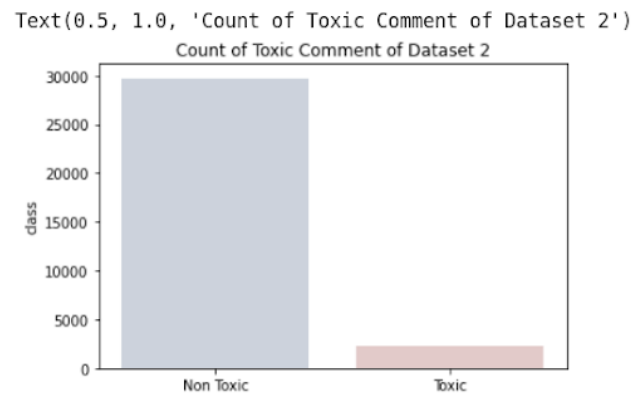

Fig. 7. Dataset 2

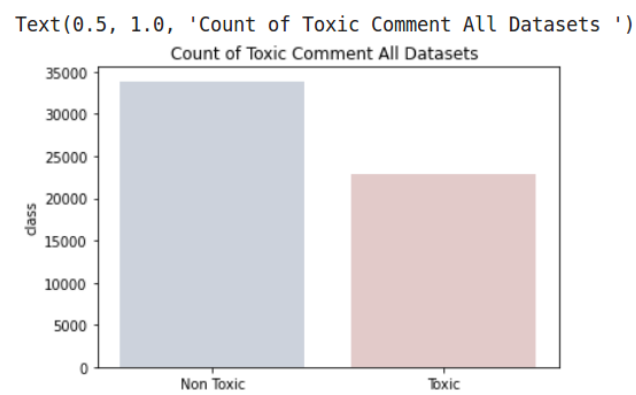

Fig. 8. Merged Dataset

2) Data Cleaning and Preprocessing: Real world data is usually in a raw and unorganized form and first needs to be cleaned in order to remove unnecessary values, fill the missing values, remove inconsistencies and make the data

Authorized licensed use limited to: IEEE Xplore. Downloaded on May 15,2021 at 08:15:50 UTC from IEEE Xplore. Restrictions apply.

more qualitative and equipped to be processed. NLTK, a Python based library, was used to perform data cleaning tasks. For every text in the dataset we performed the cleaning operation as follows:

- The short phrases were elaborated to their original forms, like “won’t” was converted to “will not”, “can’t” was converted to “can not” and so on.

- Stop words were removed from the texts so that only meaningful words remain. We observed that some of the words in the dataset that could be categorized as stop words but weren’t present as stop words in NLTK stopwords dictionary, so we created our own list of those words to filter them out from the text.

- Since the merged dataset also contained tweets it had many links, users mentions and hashtags which need to be removed as it is not useful information for text classification. Therefore we have used a regular expressions based library “re” in Python, to carry out this task.

- Next we perform lemmatization in the stop words free text, so that the words that are in different forms can be reduced to its base word, which will help to remove redundancy in terms that same word in different forms will not be considered different. Parts of speech tagging is also done to remove ambiguities in words, for eg. The word “bear” can occur as a noun and as a verb based on the context of usage. So if POS tagging isn’t used then the text can be misinterpreted. Also we converted each word into lower case so that the same word won’t be considered different just because of their case difference, during word vectorization, as they are case sensitive.

- Finally the unnecessary whitespaces and punctuations in the text were removed.

```
before cleaning: listening to sad songs on a monday morning otw to work is sad
After cleaning: listen sad songs monday morning otw work sad

before cleaning: @user #sikh #temple vandalised in in #calgary, #wso condemns  act
After cleaning: sikh  temple vandalise  calgary   wso condemn  act

before cleaning: thank you @user for you follow
After cleaning: thank  follow
```

Fig. 9. Data Cleaning

Data preprocessing is a step where the data is converted and represented in the format that is appropriate for processing. Here the class label had 2 values, either “0” for non-toxic or “1” for toxic text. But we need two neurons as output in neural networks where each neuron represents each class of text. Since there are two neurons, we need two seperate classes and not a single class with two seperate values. Rather than modifying the dataset, which is a tedious task, we used LabelEncoder class in Python to categorize and transform the values of “class” into an array of length 2. Toxic text is represented as [0,1] and non-toxic text is represented as [1,0].

3) Train and Test Splitting: While using data to train a model it is important to split the data into train and test sets. More the training data, more the deep neural network can learn and better is the accuracy. We split our data into 90% training and 10% test data.

4) Tokenization and Text to Sequence Conversion: Tokenization means division of an entire piece of text into smaller segments like paragraphs, statements, words and alphabets. Now considering paragraphs and statements as a token for text processing is a bad idea because training a neural network to understand a piece of text requires a little more basic approach. It’s similar to the scenario that if we learn a language we first learn its words and its meaning and then join the set of words into a logical sequence to form meaningful statements. If you need to learn to write, only then is it needed to learn the alphabets. So considering alphabets as a token would also be a bad idea in case of neural networks as we need it to train on words and its semantics. So we simply tokenized the text into words with the help of Keras Tokenizer class. Following is the word-to-index dictionary sample of our project:

Word	Index
oov	1
bitch	2
get	3
hoe	4
like	5
love	6
day	7
go	8
fuck	9
pussy	10
make	11
happy	12
time	13
ass	14
say	15
see	16
look	17
one	18
life	19
want	20
know	21
shit	22
good	23
people	24
today	25

Fig. 10. Word-Index Dictionary Sample

An important part here is that the Tokenizer was fit on the 30000 most frequent distinct words in train data, eliminating the lesser used words so that our embedding layer is smaller

Authorized licensed use limited to: IEEE Xplore. Downloaded on May 15,2021 at 08:15:50 UTC from IEEE Xplore. Restrictions apply.

in size. This produces a word-index dictionary that simply assigns each word a specific integer. This is important because a neural network can only train on numeric data and we need a method to convert our text data to a numeric form. The word to index dictionary generated from above is then used to convert the train and test data to sequences of numbers respectively.

```
Printing First 25 text to sequences
Statement :  wish love father father day
Sequence  :  [139, 6, 38, 38, 7]

Statement :  thank ya fuck hoser
Sequence  :  [61, 114, 9, 6212]

Statement :  ai nothin cut bitch ayeeee
Sequence  :  [40, 1258, 401, 2, 7129]

Statement :  look like move student halls master degree next year something never experience uni
Sequence  :  [17, 5, 318, 1525, 10880, 1666, 3639, 111, 118, 182, 45, 799, 3224]

Statement :  thank dom either body fat slowly decrease muscle finally decide come due workouts
Sequence  :  [61, 7130, 574, 328, 280, 2380, 8554, 4551, 165, 719, 31, 1114, 8555]

Statement :  proof good focusonthegood orlandohorror orlandoshootings praystoorlando prayfororlando
Sequence  :  [1717, 23, 16266, 4552, 10881, 16267, 720]

Statement :  thank first followers let boys
Sequence  :  [61, 73, 585, 54, 463]

Statement :  um satellite camp harbaugh happy nj new battleground paramus n j jim harbaugh
Sequence  :  [2812, 6213, 873, 8556, 12, 4983, 29, 10882, 16268, 116, 1050, 2598, 8556]

Statement :  kinda want spit gum bitch hair
Sequence  :  [984, 20, 1919, 3225, 2, 262]

Statement :  karma bitch
Sequence  :  [1398, 2]

Statement :  woohoo weeks go
Sequence  :  [2599, 374, 8]

Statement :  watch facebook page new competition post afternoon tgif fridayfeeling
Sequence  :  [66, 788, 860, 29, 2302, 273, 971, 698, 1068]

Statement :  partially glad say hi ho kermit frog slip kermit voice well
Sequence  :  [16269, 699, 15, 750, 325, 7131, 3412, 2813, 7131, 575, 97]

Statement :  arkansas gorilla simulator need adapt environment need tear city materi
Sequence  :  [3413, 554, 617, 32, 626, 589, 32, 352, 231, 2814]

Statement :  get twist shit bow bitch
Sequence  :  [3, 2486, 22, 2128, 2]
```

Fig. 11. Text to Sequence Sample

5) Word Embeddings: Word Embedding or Word Vectorization is an approach to capture the semantic relations between the different words. One hot encoding can numerically and distinctly represent the words but does not help us identify the relation between the words and its contextual meaning. In order to understand what the word really means contextually, we must understand amongst which words is it placed. Like the old saying goes "A man is known by the company he keeps", similarly the word embeddings are based on the concept that 'the context and meaning of the word can be understood by its surrounding words', and by observations it is mostly true.

In word embedding each word is a dense vector that has an array of numeric values. These numeric values display the relation strength of that word with other words. So if a word is similar to another word then its vectors must be similar and have minimal difference. For example in the context of gender, if we take vectors of king, man, woman and queen and subtract the vector of man from the vector of king and then add the vector of woman to it, then the resultant vector will be almost as same as the vector of queen. This way by numeric representation of words the word embeddings are useful in natural language processing and generation. Word embeddings are generated by training on large data and trained word embeddings are already available, like word2vec by Google, GLoVE by Stanford and SpaCy word embeddings. We have used spaCy word embeddings as it has been observed to be generally faster and have extracted the embeddings of the 30000 words that were used to fit for the tokenizer and kept our word embeddings ready.

6) Modelling Neural Network: Since neural networks need a fixed length input, the input was padded to a length of 128 words. So the input layer as depicted in the above model plot, states the input layer with 128 neurons. These 128 neurons are now connected to the embedding layer. Now we know that the input layer has input in the form of a sequence of numbers that represent the text, with the help of the word-index dictionary.

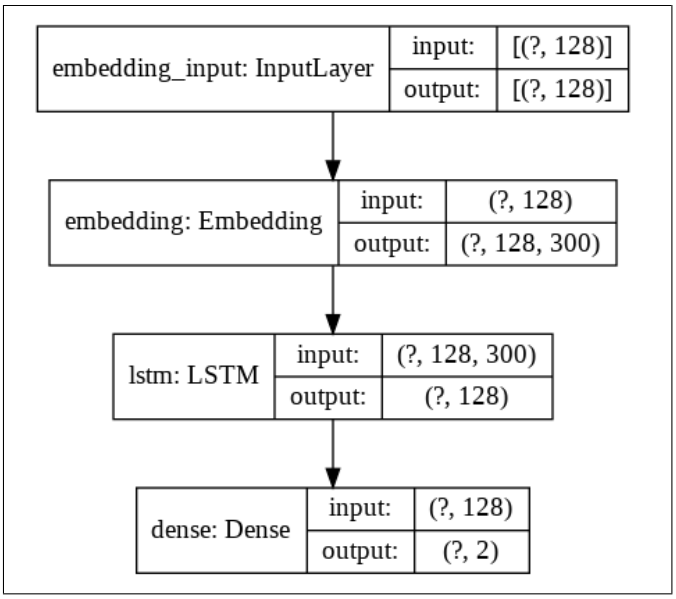

Fig. 12. Plot of Neural Network Model

The embedding layer is just a lookup table consisting of the 30000 words of our vocabulary and its corresponding word vectors, each of size 300. Now what embedding layer does is that it takes each word represented in the form of integers, from the input layer and outputs the corresponding word vector. Since each word vector has 300 values, the output of the embedding layer has 300 neurons, each neuron representing one dimension of the word vector and the weights to each neuron represent the position of word in each dimension. We have kept the weights in the Embedding layer as non trainable, because spaCy word embeddings are themselves trained to identify itself and surrounding words, appropriate context, and meanings. The fact that word vectors consist of 300 values each, represents the dimensional position of the word and is the important aspect to capture the contextual information, related words and meaning of the word. Next is the LSTM layer consisting of 128 neurons, that takes input from the 300 neurons from the previous layer, and has output size of 128. These 128 neurons are then connected to a dense layer of 2 neurons, representing the two classes of output, either toxic or non toxic. Each neuron in all the layers are densely connected with the previous and the next layers.

Authorized licensed use limited to: IEEE Xplore. Downloaded on May 15,2021 at 08:15:50 UTC from IEEE Xplore. Restrictions apply.

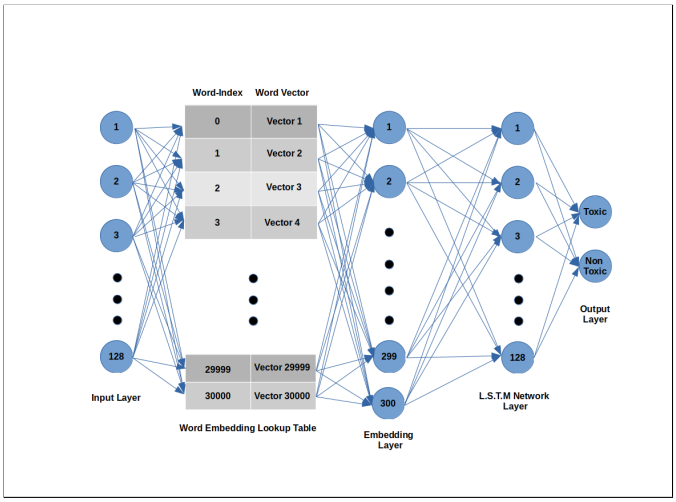

Fig. 13. Representation of Neural Network

IV. Result

The test data contains 10% of total data which is equal to 5671 values among which there are 3395 tweets labeled as non-toxic (i.e. 0) and 2276 tweets labeled as toxic(i.e. 1). We generate the confusion matrix which is represented as follows:

	Predicted 0	Predicted 1
Actual 0	TN	FP
Actual 1	FN	TP

Fig. 14. Confusion Matrix

After training the model on train data it produced following confusion matrix for the test data:

n=5671	Predicted 0	Predicted 1	
Actual 0	3272	123	n(0)=3395
Actual 1	164	2112	n(1)=2276

Fig. 15. Confusion Matrix of Test Data

Following scores can be derived from the confusion matrix:

$$Precision = \frac{TP}{TP+FP} = \frac{2112}{2112+123} = 94.49\%$$

$$Recall = \frac{TP}{TP+FN} = \frac{2112}{2112+164} = 92.79\%$$

$$Accuracy = \frac{TP+TN}{TP+FP+TN+FN}$$

$$= \frac{2112+3272}{2112+123+3272+164} = 94.94\%$$

Following confusion matrix and scores are produced for train data:

n=51035	Predicted 0	Predicted 1	
Actual 0	29571	885	n(0)=30456
Actual 1	1126	19453	n(1)=20579

Fig. 16. Confusion Matrix of Train Data

Following scores can be derived from the confusion matrix:

$$Precision = \frac{TP}{TP+FP} = \frac{19453}{19453+885} = 95.64\%$$

$$Recall = \frac{TP}{TP+FN} = \frac{19453}{19453+1126} = 94.52\%$$

$$Accuracy = \frac{TP+TN}{TP+FP+TN+FN}$$

$$= \frac{19453+29571}{19453+885+29571+1126} = 96.06\%$$

All the above scores are measured with: epoch = 5, batch_size=128 and validation_split= 0.1 (i.e. 10% test data and 90% train data) Following is the graph of model accuracy on train data and test data with respect to number of epochs:

Authorized licensed use limited to: IEEE Xplore. Downloaded on May 15,2021 at 08:15:50 UTC from IEEE Xplore. Restrictions apply.

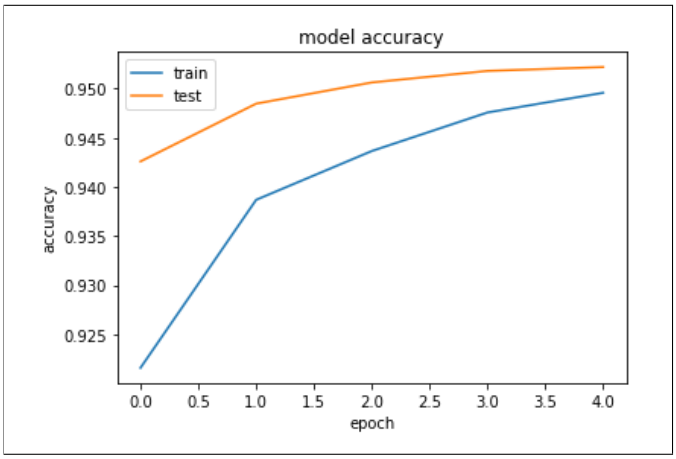

Fig. 17. Model Accuracy Plot

Following is the graph of model loss on train data and test data with respect to number of epochs:

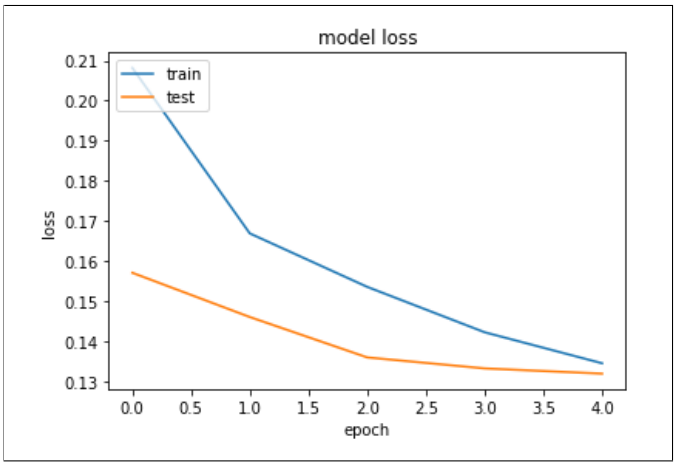

Fig. 18. Model Loss Plot

Here are some examples of classification:

1) **Input sentence:**
!!!!!!@__BrighterDays: I can not just sit up and HATE on another bitch .. I got too much shit going on!
Output:
array([[0.06137153, 0.93862844]], dtype=float32)
The output given by the model suggests that the probability of the given sentence being toxic is high and is 93.86% whereas probability of sentence being non-toxic is less and is only 6.13%. Therefore the given sentence can be considered as toxic.

1) **Input sentence:**
!!!!!!@__BrighterDays: I can not just sit up and HATE on another person .. I got too much work going on!
Output:
array([[0.9111419 , 0.08885814]], dtype=float32)
The output given by the model suggests that the probability of the given sentence being non-toxic is high and is 91.11% whereas probability of sentence being toxic is less and is only 8.85%. Therefore the given sentence can be considered as non-toxic.

V. Conclusion

As per our study on this topic we draw following conclusion:

1) LSTM neural networks are best for text classification. On providing adequate and appropriate data for training, the model provides good results on testing data.
2) The model not only classifies a given sentence as toxic or non-toxic but also gives the percentage of toxicity or non-toxicity of the given sentence.
3) We observed 94.49% precision, 92.79% recall and 94.94% Accuracy of test data on the trained model.

VI. Acknowledgment

We express our deep gratitude and regards to Prof. Sanober Sultana Shaikh, Internal Guide and Professor, Head of Department of Information Technology for her encouragement and valuable guidance in bringing shape to this dissertation. We are thankful to all the Professors and Faculty Members in the department for their teachings and academic support and thanks to Technical Staff and Non-teaching staff in the department for their support.

References

[1] 'Methodology for Twitter Sentiment Analysis', Published By : Tharsis T. P. Souza, Published on : July 2015
[2] 'Sentiment Analysis of Movie Review Using Supervised Machine Learning Techniques ' Published By: Gurshobit Singh Brar, Asst. Prof. Ankit Sharma, Published on: Nov 2018
[3] Convolutional Neural Networks for Toxic Comment Classification, Department of Computer Science and Biomedical Informatics, University of Thessally, Lamia, Greece, by Spiros V. Georgakopoulos, Sotiris K. Tasoulis, Aristidis G. Vrahatis
[4] Analysis Text of Hate Speech Detection Using Recurrent Neural Network, Computer Engineering, School of Electrical Engineering – Telkom University, Indonesia, Arum Sucia Saksesi, Muhammad Nasrun, Casi Setianingsih

Authorized licensed use limited to: IEEE Xplore. Downloaded on May 15,2021 at 08:15:50 UTC from IEEE Xplore. Restrictions apply.